Week 9

Sales and Customer Analytics System

SQL Aggregate Functions

SQL aggregate functions are functions that perform a calculation on multiple rows and return a single result

TYPES OF AGGREAGTE FUNCTIONS

| Function | Purpose | Example |
|---|---|---|
| COUNT() | Counts number of records | Total customers/orders |
| SUM() | Calculates total value | Total sales amount |
| AVG() | Finds average value | Average order value |
| MIN() | Finds minimum value | Lowest product price |
| MAX() | Finds maximum value | Highest product price |

NOTE:

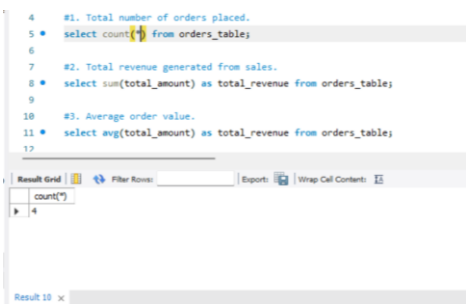
How many= use count( )

How much = use sum( )

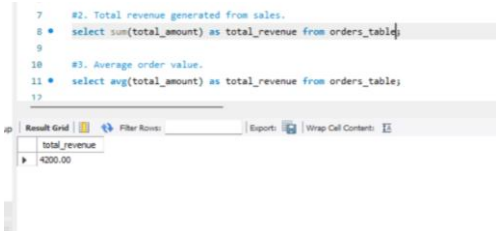
2. Generate Total Sales Reports

Tasks: Write SQL queries to calculate:

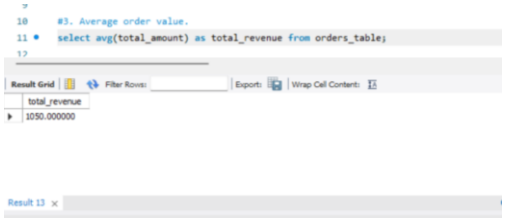
1. Total number of orders placed.




2. Total revenue generated from sales.

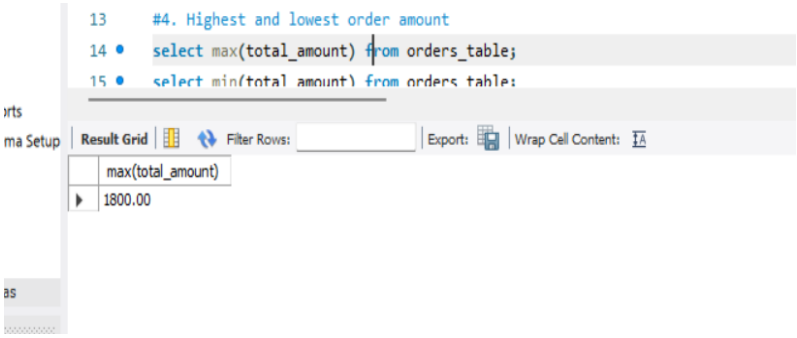



3. Average order value.

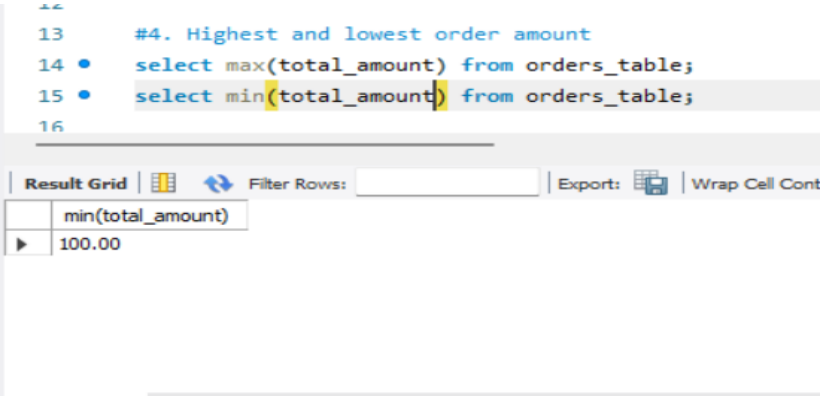



4. Highest and lowest order amount.

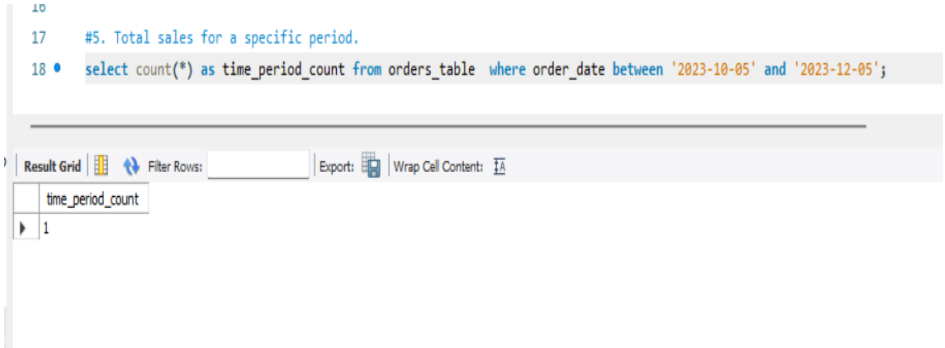
Highest:




Lowest:




5. Total sales for a specific period.



REPORT:

**Expected Report:**

| Report | Value |
|---|---|
| Total Orders | 500 |
| Total Revenue | ₹25,00,000 |
| Average Order Value | ₹5000 |

OUTPUT:

```
20 •  select count(*) as total_orders ,sum(total_amount) as total_revenue,avg(total_amount) as avg_order_value from orders_table;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| total_orders | total_revenue | avg(total_amount) |
|---|---|---|
| 4 | 4200.00 | 1050.000000 |

3. Customer Purchase Analysis Tasks: Analyze customer buying behavior.
Find:

1. Total number of orders placed by each customer.

```
26    #1. Total number of orders placed by each customer.
27 •  select count(*),customer_id  from orders_table group by customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| count(*) | customer_id |
|---|---|
| 2 | 101 |
| 1 | 102 |
| 1 | 103 |

2. Total amount spent by each customer.

```
29    #2. Total amount spent by each customer.
30 •  select sum(total_amount) as amt_spent_by_each,customer_id from orders_table group by customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| amt_spent_by_each | customer_id |
|---|---|
| 3300.00 | 101 |
| 100.00 | 102 |
| 800.00 | 103 |

Result Grid | Form Editor

3. Average spending per customer.

```
31
32      #3. Average spending per customer.
33 •    select avg(total_amount),customer_id from orders_table group by customer_id;
34
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| avg(total_amount) | customer_id |
|---|---|
| 1650.000000 | 101 |
| 100.000000 | 102 |
| 800.000000 | 103 |

Result 11 ×

Output

Action Output

4. Customers with maximum purchase amount.

```
37 •  select max(total_amount) from orders_table; # to get max values
38 •  select customer_id,total_amount as max_purchase from orders_table where total_amount=(select max(total_amount) from orders_table);
39
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| customer_id | max_purchase |
|---|---|
| 101 | 1800.00 |

5. Customers with fewer purchases.

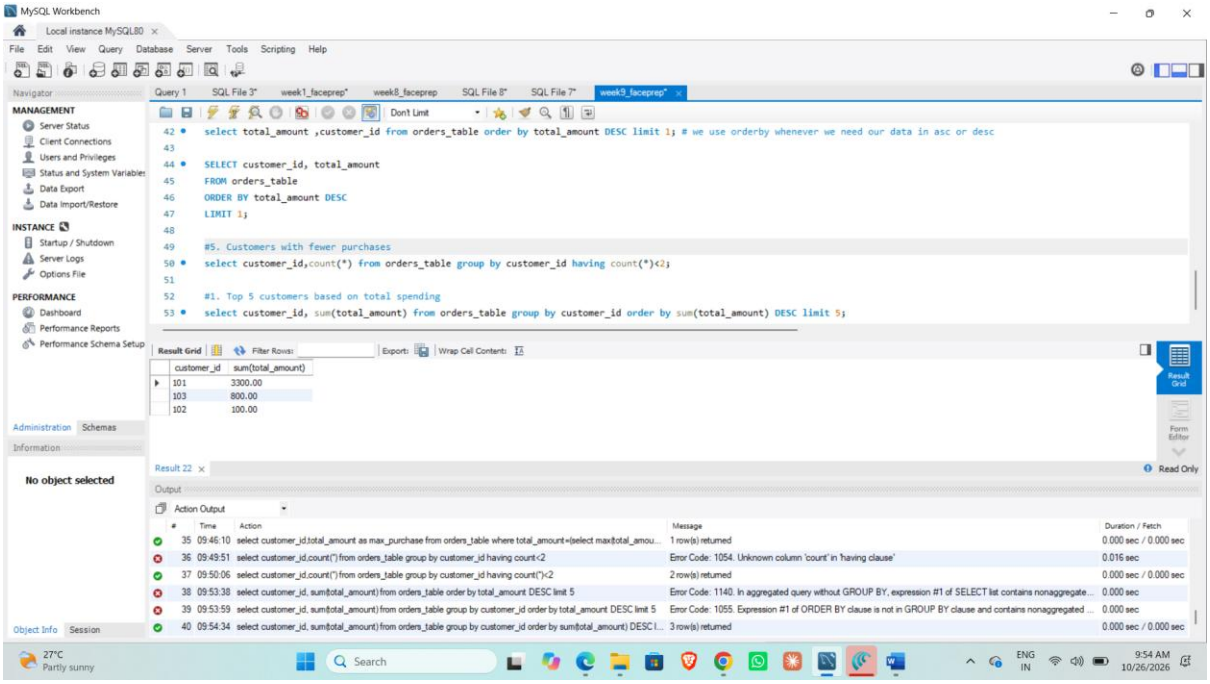
```
49      #5. Customers with fewer purchases
50 •    select customer_id,count(*) from orders_table group by customer_id having count(*)<2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

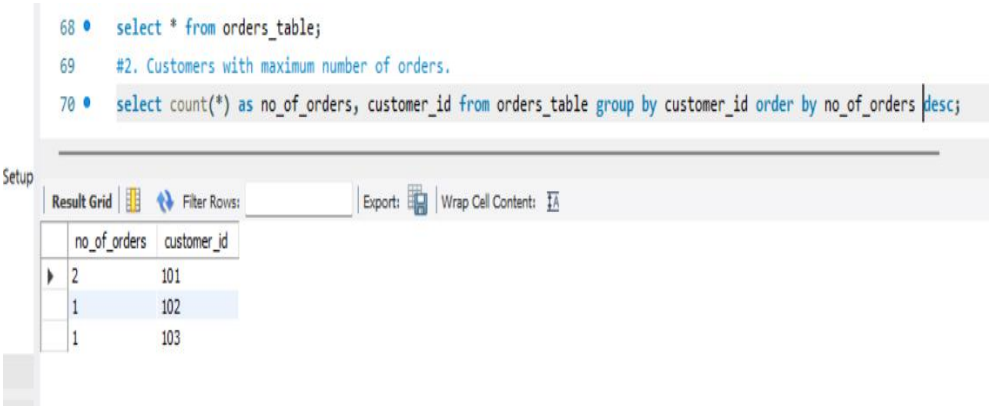
| customer_id | count(*) |
|---|---|
| 102 | 1 |
| 103 | 1 |

## 4. Find Top Customers Based on Purchase Amount

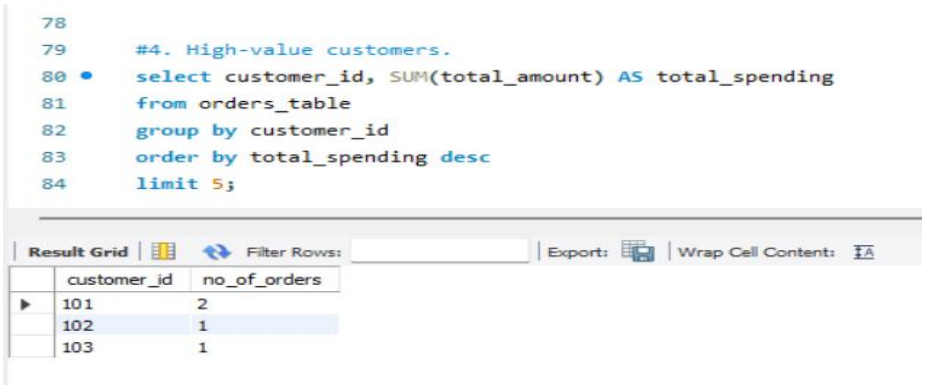
1. Top 5 customers based on total spending.




. 2. Customers with maximum number of orders.




3. Frequent customers.




4. High-value customers.

```
78
79      #4. High-value customers.
80 •    select customer_id, SUM(total_amount) AS total_spending
81      from orders_table
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| customer_id | total_spending |
|---|---|
| 101 | 3300.00 |
| 103 | 800.00 |
| 102 | 100.00 |

1. Products with maximum sales quantity.

```
86
87      #1. Products with maximum sales quantity.
88 •    show tables;
89 •    select * from product_table;
90
91 •    select product_name,quantity from product_table order by quantity desc limit 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| product_name | quantity |
|---|---|
| jeans | 12 |

2. Products generating highest revenue

```
3    #2. Products generating highest revenue.
4 •  select product_name, product_id, sum(quantity*price) as revenue from product_table group by product_id order by revenue desc limit 1;
```

ult Grid | Filter Rows: | Export: | Wrap Cell Content:

| product_name | product_id | revenue |
|---|---|---|
| dress | 133 | 20000.00 |

3. Least-selling products.

```
96      #3. Least-selling products.
97 •    select product_id,product_name ,sum(price*quantity) as selling_price from product_table group by product_id order by selling_price asc limit 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| product_id | product_name | selling_price |
|---|---|---|
| 122 | kurti | 1400.00 |

4. Products requiring promotion.

```
103 •  select product_id, COUNT(*) AS no_of_sales
104    from orders_table
105    where order_status = 'delivered'
106    group by product_id
107    order by no_of_sales ASC
108    limit 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| product_id | no_of_sales |
|---|---|
| 129 | 1 |

6. Perform Category-wise Sales Analysis Tasks: Generate reports based on product categories. Find:

1. Total sales generated by each category.

2. Number of products sold in each category.

```
110
111    #1. Total sales generated by each category.
112 •  show tables;
113 •  select * from orders_table;
114 •  select * from product_table;
115 •  select sum(o.total_amount) as total_sale,p.product_name from orders_table o join product_table p
116    on p.product_id=o.product_id group by product_name;
117
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

3. Highest revenue-generating category.

```
117
118    #3. Highest revenue-generating category.
119 •  select sum(o.total_amount) as total_sale,p.product_name from orders_table o join product_table p
120    on p.product_id=o.product_id group by product_name order by total_sale desc limit 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| total_sale | product_name |
|---|---|
| 2300.00 | kurti |

4. Average sales per category.

```
122    #4. Average sales per category.
123 •  select avg(o.total_amount) as avg_sale,p.product_name from orders_table o join product_table p
124    on p.product_id=o.product_id group by product_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| avg_sale | product_name |
|---|---|
| 1150.000000 | kurti |
| 950.000000 | toys |

7 Generate Business Analytics Reports Report

1: Sales Performance Report Display:

- Total sales
- Number of orders
- Average order value
- Highest order value

```
128    #report 1
129 •  select sum(o.total_amount) as total_sales,count(*) as no_of_orders,avg(o.total_amount),max(o.total_amount) from orders_table  o inner join product_table p  ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| total_sales | no_of_orders | avg(o.total_amount) | max(o.total_amount) |
|---|---|---|---|
| 21000.00 | 20 | 1050.000000 | 1800.00 |

Report 2:

Customer Analytics Report Display:

- Customer name
- Number of orders
- Total spending
- Purchase frequency

```
132     #report 2
133 •   select c.customer_name,count(o.order_id),sum(o.total_amount),avg(o.total_amount)
134     from customer_table c inner join orders_table o
135     on c.customer_id=o.customer_id
136     group by c.customer_id, c.customer_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| customer_name | count(o.order_id) | sum(o.total_amount) | avg(o.total_amount) |
|---|---|---|---|
| shiva | 2 | 3300.00 | 1650.000000 |
| bhiba | 1 | 100.00 | 100.000000 |
| suresh | 1 | 800.00 | 800.000000 |

Result 29

Report 3: Product Performance Report Display:

- Product name
- Quantity sold
- Revenue generated

```
144
145 •   select p.product_name,count(o.order_id) AS number_of_orders,sum(o.total_amount) AS revenue_generated
146     from product_table p inner join orders_table o on p.product_id = o.product_id
147     group by p.product_id, p.product_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| product_name | number_of_orders | revenue_generated |
|---|---|---|
| kurti | 2 | 2300.00 |
| toys | 2 | 1900.00 |

Report 4: Category Analysis Report Display:

- Category name
- Total products sold
- Total revenue

```
150     #report 4
151 •   select
152         p.category_id,
153         count(o.order_id) as total_products_sold,
154         sum(o.total_amount) as total_revenue
155     from product_table p
156     inner join orders_table o
157     on p.product_id = o.product_id
158     group by p.category_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| category_id | total_products_sold | total_revenue |
|---|---|---|
| 43 | 2 | 2300.00 |
| 49 | 2 | 1900.00 |